COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	07 / 16 / 2026	**DATE SUBMITTED:**	
NAME:	Elijah I. Dizor	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 [Arrays, Pointers and Dynamic Memory Allocation]

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and arrays

SCREENSHOT

```
#include <iostream>
#include <string>

class Student {
private:
    std::string studentName;
    int studentAge;

public:

    //constructor

    Student(std::string newName = "robin", int newAge = 22) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    }

    //deconstructor

    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }

    //Copy Constructor

    Student(const Student &copyStudent) {
```

```
    //Display Attributes
    void printDetails() {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

int main() {
    const size_t j = 5;
    std::string namesList[j] = {"Carl", "charles", "Sam", "jack", "Colin"};
    int ageList[j] = {15, 16, 18, 19, 16};

    Student studentList[j];

    for(size_t i = 0; i < j; i++) {
        studentList[i] = Student(namesList[i], ageList[i]);
    }

    for(size_t i = 0; i < j; i++) {
        studentList[i].printDetails();
    }

    return 0;
}
```

OUTPUT

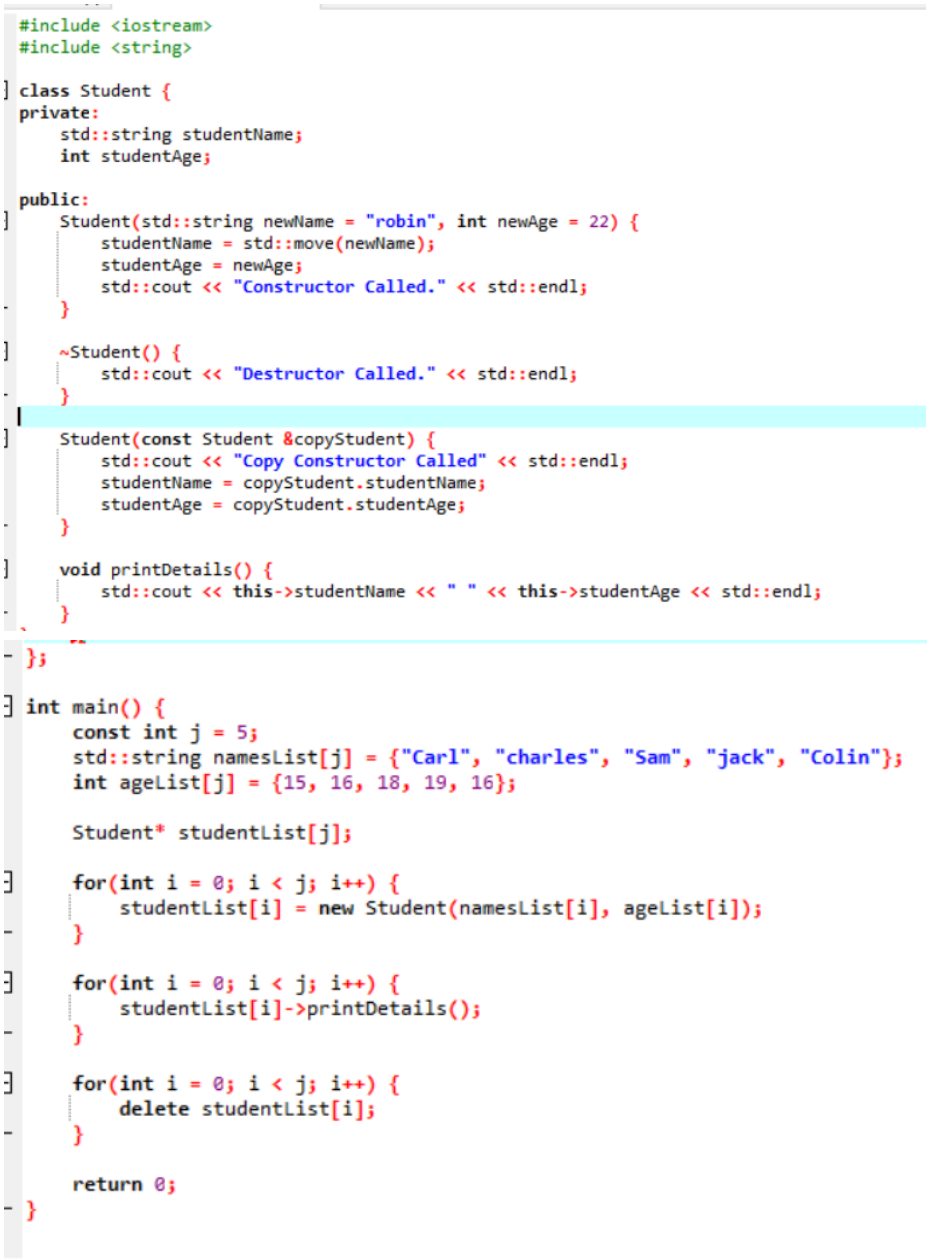
```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Carl 15
charles 16
Sam 18
jack 19
Colin 16
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.

--------------------------------
Process exited after 0.1556 seconds with return value 0
Press any key to continue . . .
```

OBSERVATION

Based on the code, declaring studentList[j]; call the constructor 5 times to fill the arrays with newName “robin” and newAge 22. It then creates 5 temporary student objects, assign them, and overwrite their initial values

Table 2 -1. Initial drive program

SCREENSHOT	
OUTPUT	LOOP A

```cpp
#include <iostream>
#include <string>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    Student(std::string newName = "robin", int newAge = 22) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    }

    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }

    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    void printDetails() {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

int main() {
    const int j = 5;
    std::string namesList[j] = {"Carl", "charles", "Sam", "jack", "Colin"};
    int ageList[j] = {15, 16, 18, 19, 16};

    Student* studentList[j];

    for(int i = 0; i < j; i++) {
        studentList[i] = new Student(namesList[i], ageList[i]);
    }

    for(int i = 0; i < j; i++) {
        studentList[i]->printDetails();
    }

    for(int i = 0; i < j; i++) {
        delete studentList[i];
    }

    return 0;
}
```

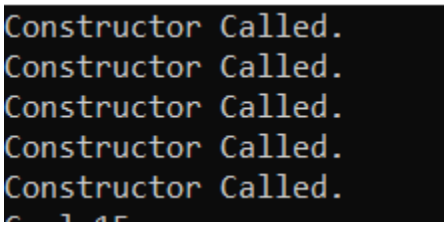
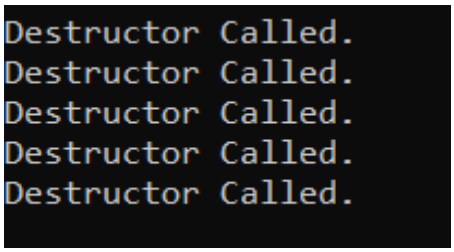
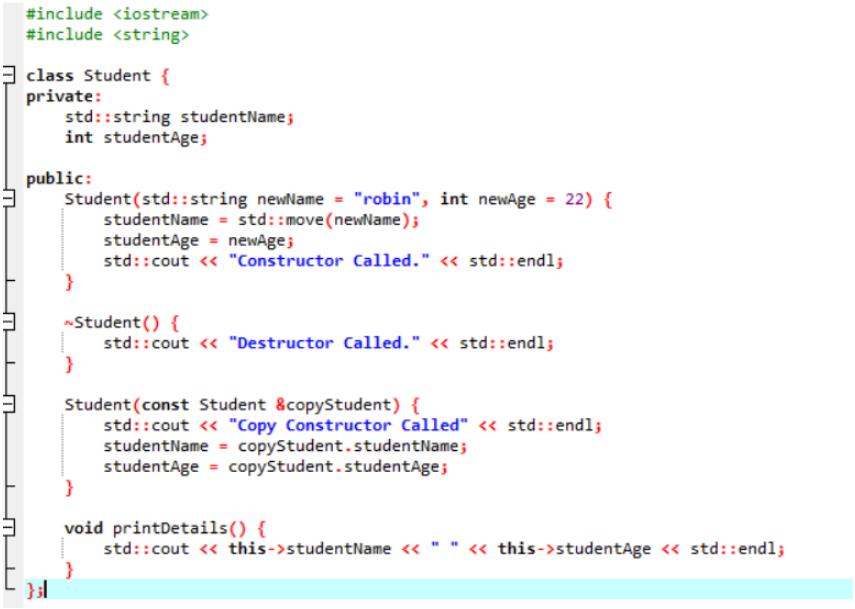
	 LOOP B 
OBSERVATION	Changing the studentList prevents the default newName and newAge objects from being created. The 'new' keyword input allocates each student directly in heap memory once, while the constructor gets called 5 times.

Table 2-2. Modified driver program with student lists

SCREENSHOT	

```cpp
#include <iostream>
#include <string>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    Student(std::string newName = "robin", int newAge = 22) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    }

    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }

    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    void printDetails() {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};
```

```
int main() {
    const int j = 5;
    std::string namesList[j] = {"Carl", "charles", "Sam", "jack", "Colin"};
    int ageList[j] = {15, 16, 18, 19, 16};

    Student* studentList = new Student[j];

    for(int i = 0; i < j; i++) {
        studentList[i] = Student(namesList[i], ageList[i]);
    }

    for(int i = 0; i < j; i++) {
        studentList[i].printDetails();
    }

    delete[] studentList;

    return 0;
}
```

OUTPUT

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Carl 15
charles 16
Sam 18
jack 19
Colin 16
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.

--------------------------------
Process exited after 0.1577 seconds with return value 0
Press any key to continue . . .
```

OBSERVATION

The code allocates a dynamic array directly on the heap using new Student[j]. It runs the default constructor 5 times, while creating 5 robin placeholders. Inside the temporary loop Student, objects are created and copied over the heap array elements by using the default assignment operator.

Table 2-3. Final drive program

OBSERVATION	

Table 2-4. Modifications / corrections necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **chatgpt**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance and explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used ai to give explanation of how the code works that is easy for me to understand a little bit, since i used dev c++ and not vscode with c++ extensions, i had to find certain ways to make some commands work such as 'std::move' on a outdated version of dev c++**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

"I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own."